docs-ari-strategy

ARI Adoption Strategy — How a spec becomes a standard

ARI Adoption Strategy — How a spec becomes a standard

This document is the playbook for turning **ARI** (`../ARI-SPEC.md`) from a doc in one repo into an interface the industry targets. It is modeled on how durable infrastructure standards actually won (OpenStack, the Open Compute Project, POSIX, JDBC, the Language Server Protocol) rather than on how products launch.

> **The thesis in one line:** a standard wins by claiming a layer the incumbents *structurally cannot follow you into*, proving it with a reference implementation you dogfood, and making conformance *testable* so others can credibly say "we implement it."

---

1. Why this can work (the structural bet)

The mature agent frameworks — LangGraph, CrewAI, AutoGen — are Python-locked by design. That is fine for server apps, but it means they **cannot** serve:

- embodied / robotic systems (real-time, no GC pauses),
- edge / on-device deployments (constrained, often offline, local models),
- agents embedded inside native apps (game engines, DSP, trading, CAD).

ARI's **ARI-Embedded** profile is defined precisely around requirements those runtimes cannot meet without ceasing to be Python-locked (native core, no hot-path allocation, bounded memory). That is the moat. We are not trying to be "a faster LangGraph" on LangGraph's turf — a losing standards fight. We are defining the lane they can't enter.

This mirrors the Cole Crawford pattern: pick the layer defined by a hard constraint (for him, the physics of edge latency/location; for us, the runtime constraints of embedded/embodied AI), then own both the **standard** and its **reference implementation**.

---

2. The three artifacts a standard needs

A standard is not a marketing claim; it is three things that must all exist:

Artifact	Status	Where
The spec — normative contract with MUST/SHOULD/MAY	drafted	`ARI-SPEC.md`
A reference implementation — proof the spec is buildable	exists	`agentcore` (this repo)
A conformance kit — lets a *third party* prove they comply	to build	`ari-conformance/` (proposed)

The third is what most "standards" are missing and why they stay vanity docs. Browsers got real because of the Acid tests; OpenStack interop got real because of RefStack. **Build the conformance kit early** — it is the thing that lets someone else say "ARI-Core + ARI-Embedded conformant" with evidence, and it is what makes the spec falsifiable rather than aspirational.

Conformance kit, concretely

- A language-neutral test corpus: JSON fixtures + expected behaviors for each numbered requirement in the spec (the Appendix B checklist becomes executable).
- A thin per-language harness that drives a runtime through the corpus.
- A published results format ("ARI 0.1 · Core+Embedded · kit v0.1 · pass 47/47").
- Run `agentcore` against it in CI so the reference implementation is provably conformant and regressions are caught.

---

3. The adoption ladder (dogfood → niche → ecosystem)

Do not chase external adopters first. Earn the right to be a standard by proving it on systems you control, then expand outward one credible step at a time.

Rung 1 — Dogfood in your own embodied system (you control the believer). Make ARI the runtime contract that **Tilo** (Unitree H2) actually runs on. This is the lighthouse deployment: a real humanoid running an ARI-Embedded runtime is worth more than any benchmark. It also forces the spec to be honest — anything that doesn't survive contact with a real robot gets fixed.

Rung 2 — Second independent proof point in your own portfolio. Embed an ARI runtime in a *latency-sensitive, non-robotic* native context — the trading stack is the obvious candidate. Two independent deployments across two domains, both yours, is the credibility floor before asking anyone external to care.

Rung 3 — A second *implementation*, not just a second user. A standard with one implementation is a library. Encourage (or write) a second conformant runtime — e.g. a Rust ARI-Core core — even a partial one. The moment two runtimes pass the same conformance kit, "interface" stops being a metaphor.

Rung 4 — Adapters that make ARI the path of least resistance. Ship adapters so ARI sits *with* the ecosystem, not against it:

- expose an ARI runtime's tools over **MCP**,
- let an ARI runtime participate in an **A2A** mesh,
- provide a shim so existing LangGraph/CrewAI tool definitions can be invoked through an ARI ToolRegistry. Being MCP/A2A-complementary (spec §1.3) means you ride the protocol wave instead of fighting it.

Rung 5 — Neutral governance. Once there is a second implementation and outside interest, move the spec out of this repo into a neutral home (its own `ari-spec` repo/org, CC BY 4.0 text, an open issue/RFC process). Standards die when they look like one company's property. Cole's standards (OpenStack, OCP) all moved to foundations precisely to shed that perception. Keep `agentcore` as the reference implementation, but let the *spec* belong to everyone.

---

4. Positioning rules (how to talk about it)

- **Lead with embodied/edge, not speed.** "The agent runtime you can embed in a robot or edge device" — not "a faster LangGraph." Speed is a feature; the embedded lane is the identity.
- **Always name the layer.** ARI is the *runtime* layer beneath MCP/A2A. Repeat the stack diagram everywhere. Owning a clear layer is how people know where to put you.
- **Be honest about maturity.** The reference implementation is early; say so. Credibility compounds; overclaiming burns it once.
- **Conformance language is disciplined.** Never "ARI-compatible" alone — always "ARI-Core" / "ARI-Embedded" + version. Sloppy conformance claims are how a standard's meaning erodes.

---

5. Naming, trademark, and the spec/impl split

- **ARI** = the standard (the interface). **agentcore** = the reference implementation. Keep these rigorously distinct in all materials — conflating them makes ARI look like a single vendor's API and kills neutral adoption.
- ⚠ **Naming collision — resolve before any go-to-market push.** "AgentCore" is now **Amazon Bedrock AgentCore** (AWS; GA 2025-10-13), which itself ships an "AgentCore Runtime" — the *same layer this project targets*. This is a direct name + semantic collision with a major cloud vendor: it will lose every search, invites trademark risk, and an investor/operator will flag it on slide one. **Recommendation:** keep **ARI** (the standard name is defensible) and **rename the implementation** to something ownable before evangelizing. The standard surviving a rename of its reference implementation is exactly why ARI and agentcore are kept separate (above). Treat this as a blocking pre-launch task.
- Reserve the obvious homes early (an `ari-spec` repo, a docs domain) so the standard has a neutral-looking address when Rung 5 arrives.
- Spec text under **CC BY 4.0** (anyone can implement); reference code under **Apache-2.0** (patent grant, embeddable in proprietary products). This split is deliberate and is stated in spec §13.

---

6. The 90-day concrete next steps

1. **Merge the spec + repositioned README** so the repo reads as "reference implementation of ARI," not "another agent framework." *(this change)*
2. **Stand up `ari-conformance/`** with the Appendix B checklist as executable tests; wire it into CI against `agentcore`.
3. **Port Tilo's agent loop onto an ARI-Embedded build** of agentcore; capture the gaps the real robot exposes and fold them into ARI 0.2.
4. **Write the MCP adapter** (expose ARI tools over MCP) — the cheapest way to be seen as ecosystem-complementary.

5. **Publish ARI 0.1 as an RFC** (issue + discussion), inviting one external reviewer from the embedded/robotics world.

The order matters: spec + reference impl + conformance kit + a real embodied deployment, *before* any push for outside adoption. That sequence is what turns "a doc I wrote" into "the interface other people target."